

ESPRIT — PROJET INTÉGRATION 4SAE9

Guide de Validation MLA

Prédiction de Réussite des Étudiants
Machine Learning Avancé · CRISP-DM

PCA

K-Means

Random Forest

XGBoost

Cross Validation

Zeineb Necib | Groupe 6 | 2025–2026

Dataset : UCI Student Performance · FastAPI + Angular + Spring Boot

TRÈS IMPORTANT : Ce guide te prépare à expliquer ORALEMENT chaque algorithme et chaque décision. Ne mémorise pas — comprends pourquoi chaque choix a été fait.

Table des matières

1. **1.** Architecture globale — ML + Web
2. **2.** Dataset & Méthodologie CRISP-DM
3. **3.** Préparation des données
4. **4.** Réduction de dimensionnalité — ACP (PCA)
5. **5.** Apprentissage non supervisé — K-Means
6. **6.** Apprentissage supervisé — Random Forest
7. **7.** Apprentissage supervisé — XGBoost **[NOUVEAU]**
8. **8.** Validation croisée — Cross Validation k=5 **[NOUVEAU]**
9. **9.** API FastAPI déployée & intégration web
10. **10.** Questions-Réponses pour la soutenance **[CRITIQUE]**
11. **11.** Checklist finale avant validation

1

Architecture Globale — ML + Web

Le ML est un microservice Python indépendant (FastAPI) intégré dans l'architecture Spring Cloud existante. Le frontend Angular appelle l'API Gateway qui route vers le ML Service.

Stack complète

Couche	Technologie	Port	Rôle
Frontend	Angular 17	4200	Interface étudiant/admin, affichage des prédictions
API Gateway	Spring Cloud Gateway	8080	Point d'entrée unique, routage vers ML Service
ML Service	FastAPI (Python)	5000	Prédiction, clustering, exposition des modèles
Claim Service	Spring Boot	8081	Gestion des réclamations
Auth Service	Spring Boot	8083	JWT, utilisateurs
Eureka Server	Spring Cloud Netflix	8761	Service Discovery

Flux de prédiction (Angular → ML Service)



Endpoints ML disponibles

Méthode	Endpoint	Description
GET	/health	Statut du service
GET	/model-info	Métriques du meilleur modèle déployé
POST	/predict	Prédiction pass/fail avec le meilleur modèle (sélectionné par CV)
POST	/predict/rf	Prédiction avec Random Forest spécifiquement
POST	/predict/xgb	Prédiction avec XGBoost spécifiquement
POST	/cluster	Profil étudiant (At-Risk / Average / High-Performing)
POST	/analyze	Prédiction + profil en un seul appel

Q : Pourquoi FastAPI et pas Spring Boot pour le ML ?

R : Parce que l'écosystème Python (scikit-learn, XGBoost, pandas, numpy) est le standard pour le Machine Learning. Spring Boot ne dispose pas de ces bibliothèques. FastAPI est léger, rapide, et produit automatiquement une documentation Swagger. Il s'intègre parfaitement comme microservice dans l'architecture Spring Cloud via l'API Gateway.

2 Dataset & Méthodologie CRISP-DM

Dataset — UCI Student Performance (student-mat.csv)

395

ÉTUDIANTS

33FEATURES
TOTALES**17**

CATÉGORIELLES

16

NUMÉRIQUES

67.1%TAUX DE
RÉUSSITE**0**VALEURS
MANQUANTES**Variable cible : G3 (note finale)**

G3 est la note de l'examen final (0–20). On la binarise : $G3 \geq 10$ = **réussite (1)**, $G3 < 10$ = **échec (0)**.

Features clés : **G1** (1er trim.), **G2** (2ème trim.), **failures** (échecs passés), **studytime**, **absences**.

Méthodologie CRISP-DM appliquée

- 1 Business Understanding** : Détecter les étudiants à risque pour intervention pédagogique ciblée
- 2 Data Understanding** : Analyse descriptive, corrélations, distribution de G3
- 3 Data Preparation** : Encodage LabelEncoder + StandardScaler + binarisation cible
- 4 Modeling** : PCA → K-Means → Random Forest → XGBoost → Cross Validation
- 5 Evaluation** : Accuracy, F1-Score, ROC-AUC, Silhouette Score
- 6 Deployment** : FastAPI + intégration Angular via API Gateway

Q : Pourquoi ce dataset ? Quel est le lien avec le projet ?

R : Ce dataset de l'UCI contient des données réelles sur 395 étudiants portugais — notes, absences, contexte familial. Le lien avec le projet est direct : notre plateforme éducative gère des demandes de rattrapage. Prédire si un étudiant va réussir ou échouer AVANT l'examen de rattrapage permet aux enseignants d'intervenir en amont. C'est une contribution concrète à l'ODD 4 (Éducation de Qualité).

Q : Pourquoi binariser G3 au lieu de prédire la note directement ?

R : Parce que la question métier est binaire : "l'étudiant va-t-il réussir ou échouer ?". La régression sur G3 donnerait une note précise mais peu actionnable. La classification binaire (pass/fail) est directement exploitable : si "fail", déclencher une alerte. De plus, les métriques de classification (F1, AUC) sont plus adaptées à la détection d'un phénomène rare (l'échec représente 32.9% des cas).

3 Préparation des Données

Pipeline de prétraitement

 backend/ml-service/app/train.py – fonction preprocess()

```
# 1. Encodage des variables catégorielles
for col in CAT_COLS:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    encoders[col] = le      # sauvegardé pour l'inférence

# 2. Cible binaire
df['pass'] = (df['G3'] >= 10).astype(int)

# 3. StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

LabelEncoder

Transforme les valeurs texte en entiers : "GP"→0, "MS"→1 / "M"→1, "F"→0. Appliqué sur 17 colonnes (school, sex, address, Mjob, Fjob, guardian...). Les encodeurs sont sauvegardés en `.pkl` pour être réutilisés à l'inférence.

StandardScaler

Centre chaque feature : $\mu=0$, $\sigma=1$. Nécessaire pour PCA (sensible aux échelles) et K-Means (distances euclidiennes). Sans ça, une feature comme `absences` (0–75) dominerait sur `studytime` (1–4).

Q : Pourquoi utiliser LabelEncoder et pas OneHotEncoder ?

R : LabelEncoder transforme chaque catégorie en un entier. OneHotEncoder crée une colonne binaire par catégorie. Avec 17 variables catégorielles ayant jusqu'à 5 valeurs chacune, OneHotEncoder aurait créé ~50 colonnes supplémentaires. On utilise LabelEncoder car les algorithmes suivants (RandomForest, XGBoost) gèrent bien les entiers ordinaux, et la PCA réduit ensuite la dimensionnalité de toute façon.

Q : Pourquoi normaliser avec StandardScaler ?

R : La PCA maximise la variance — si une feature a une grande variance par nature (ex: absences 0-75), elle va dominer les composantes principales. StandardScaler ramène tout à la même échelle ($\mu=0$, $\sigma=1$) pour que chaque feature contribue équitablement. K-Means utilise des distances euclidiennes — sans normalisation, les features à grande échelle biaisent les clusters.

4 Réduction de Dimensionnalité — ACP (PCA)

La PCA transforme les 32 features corrélées en composantes indépendantes, en gardant celles qui expliquent 95% de la variance.

32

FEATURES INITIALES

27COMPOSANTES
RETENUES**95.1%**

VARIANCE EXPLIQUÉE

-16%RÉDUCTION
DIMENSIONNALTÉ backend/ml-service/app/train.py

```
pca = PCA(n_components=0.95, random_state=42)
X_pca = pca.fit_transform(X_scaled)
# n_components=0.95 → sklearn choisit automatiquement le nombre
# de composantes pour couvrir 95% de la variance
```

Pourquoi 95% et pas 99% ou 100% ?

95% est un compromis standard : on élimine ~5% de variance correspondant souvent au bruit. Garder 100% annulerait l'intérêt de la PCA. À 95%, on passe de 32 à 27 composantes — une réduction modeste mais qui supprime la corrélation entre features (G1 et G2 sont très corrélées — la PCA les fusionne dans une composante).

Q : Qu'est-ce que la PCA fait concrètement ?

R : La PCA cherche les directions (composantes principales) qui maximisent la variance dans les données. PC1 capture la direction de plus grande variance, PC2 la deuxième, etc. Ces composantes sont orthogonales (non corrélées). En gardant seulement les 27 premières, on compresse les données tout en préservant 95.1% de l'information. Concrètement, des features corrélées comme G1 et G2 sont fusionnées en une seule composante.

Q : Pourquoi appliquer la PCA AVANT K-Means et Random Forest ?

R : Deux raisons : (1) **K-Means** souffre de la malédiction de la dimensionnalité — les distances euclidiennes deviennent peu informatives en haute dimension. Réduire la dimensionnalité améliore la qualité des clusters. (2) **Random Forest** — la PCA élimine le bruit et la redondance, ce qui peut améliorer la généralisation. De plus, les deux algorithmes travaillent sur les mêmes représentations, ce qui assure la cohérence.

Q : Peut-on interpréter les composantes PCA ?

R : Pas directement — c'est une limite connue de la PCA. Une composante est une combinaison linéaire de toutes les features originales. PC1 pourrait correspondre à "performance académique globale" (fort poids sur G1, G2), PC2 à "comportement" (poids sur absences, Dalc), mais ce n'est qu'une interprétation. Pour la validation, on dit que la PCA est utilisée comme outil de prétraitement, pas d'interprétation.

5 Apprentissage Non Supervisé — K-Means

K-Means est un algorithme de clustering : il regroupe les étudiants en k groupes homogènes SANS utiliser les labels de réussite/échec (non supervisé).

k=3

CLUSTERS

0.0655

SILHOUETTE SCORE

Elbow

MÉTHODE SÉLECTION K

n_init=10

INITIALISATIONS

Les 3 profils identifiés

Cluster	Label	G1 moy.	G3 moy.	Échecs	Interprétation
0	At-Risk	12.79	12.90	0.06	Bons résultats mais nécessitent surveillance
1	Average	8.67	7.78	0.97	Résultats faibles, échecs passés — priorité soutien
2	High-Performing	9.46	8.32	0.38	Profil intermédiaire bon

Pourquoi k=3 ?

Choisi par la **méthode Elbow** (coude sur la courbe d'inertie) et confirmé par le Silhouette Score. k=3 a aussi une justification métier : 3 profils d'étudiants = à risque, moyen, fort. C'est exploitable pédagogiquement.

Q : Comment fonctionne K-Means ? Décris l'algorithme.

R : 1. Initialisation : k centroïdes placés aléatoirement (n_init=10 pour éviter les mauvais départs). 2. Attribution : chaque point est assigné au centroïde le plus proche (distance euclidienne). 3. Mise à jour : chaque centroïde devient la moyenne de ses points assignés. 4. Répétition jusqu'à convergence (les centroïdes ne bougent plus). L'objectif est de minimiser l'inertie (somme des distances au centroïde).

Q : Qu'est-ce que le Silhouette Score de 0.0655 indique ?

R : Le Silhouette Score mesure à quel point un point est similaire à son cluster par rapport aux autres clusters. Il varie de -1 à +1 : proche de 1 = clusters très séparés, proche de 0 = clusters qui se chevauchent. 0.0655 est faible, ce qui indique que les profils étudiants ne sont pas très séparés dans l'espace PCA — les étudiants "at-risk" et "average" ont des profils similaires. C'est une limite du dataset et non de l'algorithme.

Q : Quelle est la différence entre supervisé et non supervisé ? Pourquoi K-Means ici ?

R : Supervisé = on a des étiquettes (pass/fail) pour entraîner le modèle. Non supervisé = pas d'étiquettes, l'algorithme trouve lui-même la structure. K-Means est utilisé ici pour découvrir des groupes d'étudiants naturels SANS qu'on lui dise d'avance qui réussit. Cela permet de créer des profils exploitables même pour de nouveaux étudiants sans historique de résultats.

6 Apprentissage Supervisé — Random Forest

Random Forest est un algorithme d'ensemble : il combine 100 arbres de décision pour faire une prédiction robuste.

81.0%

ACCURACY

0.867

F1-SCORE

0.883

ROC-AUC

100

N° ARBRES

80/20

TRAIN/TEST SPLIT

 backend/ml-service/app/train.py

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
y_proba = rf.predict_proba(X_test)[:, 1] # probabilité de réussite
```

Résultats sur le test set (79 étudiants)

Sur 26 étudiants en échec réel : 15 bien détectés (recall=58%). Sur 53 en réussite réelle : 49 bien détectés (recall=92%). Le modèle est meilleur pour prédire la réussite que l'échec — logique vu le déséquilibre des classes (67% de réussite).

Q : Comment fonctionne Random Forest ?

R : Random Forest crée 100 arbres de décision, chacun entraîné sur un sous-échantillon aléatoire des données (bootstrap) et avec un sous-ensemble aléatoire des features à chaque nœud. La prédiction finale est un vote majoritaire des 100 arbres. Cette double randomisation réduit la variance et évite l'overfitting par rapport à un arbre de décision unique.

Q : Pourquoi F1-Score plutôt qu'Accuracy pour évaluer ?

R : L'Accuracy est trompeuse sur des classes déséquilibrées : un modèle qui prédit "toujours réussite" aurait 67% d'accuracy sans rien apprendre. Le F1-Score est la moyenne harmonique de la précision et du rappel — il pénalise les faux négatifs (étudiants en échec non détectés) et les faux positifs. Le ROC-AUC mesure la capacité à discriminer les deux classes pour tous les seuils de décision.

7 Apprentissage Supervisé — XGBoost

XGBoost (eXtreme Gradient Boosting) est un algorithme de boosting : il construit les arbres séquentiellement, chaque arbre corrigeant les erreurs du précédent.

~83.5%

ACCURACY
CV

~0.889

F1-SCORE CV

~0.897

ROC-AUC CV

100

N°
ESTIMATEURS

0.1

LEARNING
RATE

6

MAX DEPTH

 backend/ml-service/app/train.py

```
xgb = XGBClassifier(
    n_estimators=100,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,           # 80% des données par arbre
    colsample_bytree=0.8,    # 80% des features par arbre
    eval_metric='logloss',
    random_state=42,
    verbosity=0
)
xgb.fit(X_train, y_train)
```

Différence Random Forest vs XGBoost

RF : Arbres construits en *parallèle*, chacun indépendant. Vote majoritaire. Robuste au bruit.

XGBoost : Arbres construits en *séquence*. Chaque arbre corrige les résidus du précédent. Plus puissant mais plus sensible aux hyperparamètres.

Hyperparamètres clés

learning_rate=0.1 : pas d'apprentissage — faible = plus robuste mais plus lent.

max_depth=6 : profondeur max de chaque arbre — contrôle l'overfitting.

subsample=0.8 + colsample_bytree=0.8 : régularisation par stochastique.

Q : Comment fonctionne le gradient boosting (XGBoost) ?

R : XGBoost construit les arbres séquentiellement. Le premier arbre prédit à partir des données brutes. Le deuxième arbre est entraîné sur les RÉSIDUS (erreurs) du premier. Le troisième corrige les erreurs du deuxième, et ainsi de suite. La prédiction finale est la somme pondérée de tous les arbres, avec un learning_rate qui contrôle la contribution de chaque arbre. L'objectif est minimisé par descente de gradient — d'où "gradient boosting".

Q : Pourquoi ajouter XGBoost si Random Forest marche déjà bien ?

R : Deux raisons : (1) **Performance** : XGBoost obtient généralement de meilleures métriques que RF car il optimise directement l'objectif. Sur ce dataset, il améliore le ROC-AUC de ~ 0.883 à ~ 0.897 . (2) **Rigueur scientifique** : le prof demande plusieurs algorithmes pour comparer. On ne choisit pas un modèle sans comparer — la cross-validation détermine objectivement lequel est meilleur et c'est ce modèle qui est déployé.

Q : Que fait `subsample=0.8` dans XGBoost ?

R : Chaque arbre est entraîné sur un sous-échantillon aléatoire de 80% des données (sans remise). Cela introduit de la stochasticité comme dans Random Forest, ce qui réduit l'overfitting. `colsample_bytree=0.8` fait la même chose sur les features : chaque arbre n'utilise que 80% des features disponibles.

8 Validation Croisée — Cross Validation k=5

La cross-validation évalue la généralisation d'un modèle de façon plus fiable qu'un simple train/test split, en utilisant TOUTES les données pour l'évaluation.

Comment fonctionne StratifiedKFold k=5

- 1 Diviser les 395 étudiants en 5 folds égaux (~79 chacun)
- 2 **Stratifié** : chaque fold contient la même proportion réussite/échec (67%/33%) que le dataset global
- 3 5 itérations : à chaque fois, 1 fold = test, 4 folds = train
- 4 Score final = **moyenne ± écart-type** des 5 scores

 backend/ml-service/app/train.py

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rf_auc_cv = cross_val_score(
    RandomForestClassifier(n_estimators=100, random_state=42),
    X_pca, y, cv=cv, scoring='roc_auc'
).mean()

xgb_auc_cv = cross_val_score(
    XGBClassifier(n_estimators=100, learning_rate=0.1, ...),
    X_pca, y, cv=cv, scoring='roc_auc'
).mean()

# Sélection automatique du meilleur modèle
best_clf = xgb if xgb_auc_cv >= rf_auc_cv else rf
```

Résultats comparatifs (CV k=5)

Modèle	Accuracy CV	F1-Score CV	ROC-AUC CV	Déployé ?
Random Forest	0.827 ± 0.022	0.884 ± 0.018	0.891 ± 0.025	—
XGBoost	0.835 ± 0.019	0.889 ± 0.016	0.897 ± 0.021	✓ Meilleur

Q : Pourquoi la cross-validation est plus fiable qu'un simple train/test split ?

R : Un simple split 80/20 dépend du hasard de la partition : si les 20% de test contiennent des cas faciles, l'accuracy sera surestimée. Avec CV k=5, chaque étudiant passe UNE FOIS en test — on évalue vraiment la généralisation sur toutes les données. L'écart-type $\pm$ mesure la stabilité : ± 0.021 sur le ROC-AUC de XGBoost signifie que le modèle est consistant à travers les 5 folds.

Q : Pourquoi "Stratifié" (StratifiedKFold) et pas un KFold simple ?

R : Parce que les classes sont déséquilibrées (67% réussite, 33% échec). Un KFold simple pourrait créer un fold avec 80% de réussites et un autre avec 50% — les scores seraient incomparables. StratifiedKFold garantit que chaque fold a la même distribution que le dataset global. C'est la bonne pratique pour les problèmes de classification déséquilibrée.

Q : Pourquoi utiliser ROC-AUC comme critère de sélection du meilleur modèle ?

R : Le ROC-AUC mesure la capacité de discrimination globale du modèle pour TOUS les seuils de décision (pas seulement le seuil 0.5 par défaut). C'est la métrique la plus robuste pour comparer deux classifieurs sur un problème déséquilibré. Un AUC de 0.897 signifie que XGBoost distingue correctement réussite/échec dans 89.7% des cas. Si on utilisait l'Accuracy, un modèle biaisé vers la classe majoritaire (réussite) pourrait sembler meilleur sans vraiment détecter les échecs.

9 API FastAPI & Intégration Web

Démarrage du service (Docker)

```
docker-compose up ml-service
# Au démarrage : ensure_model() entraîne RF + XGBoost si les .pkl n'existent pas
# Charge ensuite scaler.pkl, pca.pkl, kmeans.pkl, rf.pkl, xgb.pkl, best_classifier.pkl
```

Exemple d'appel à /predict

```
# POST http://localhost:5000/predict
{
  "school": "GP", "sex": "M", "age": 17,
  "G1": 8, "G2": 7, "failures": 2,
  "studytime": 1, "absences": 15, ...
}

# Réponse :
{
  "prediction": "fail",
  "probability_pass": 0.1823,
  "probability_fail": 0.8177,
  "model_used": "xgboost"
}
```

Pipeline de prédiction (inférence)

 backend/ml-service/app/main.py – fonction `_to_pca_vector()`

```
# 1. LabelEncoder → encode les valeurs texte
# 2. StandardScaler → normalise
# 3. PCA → réduit à 27 composantes
# 4. best_classifier.predict() → pass ou fail
```

Sélection automatique du meilleur modèle

À chaque entraînement, `train.py` compare le ROC-AUC CV de RF et XGBoost. Le gagnant est sauvegardé dans `best_classifier.pkl`. `POST /predict` utilise toujours ce meilleur modèle. Le nom du modèle actif est visible dans `GET /model-info` → champ `best_classifier`.

Intégration Angular

Le frontend Angular appelle `POST /api/ml/predict` via l'API Gateway (port 8080) qui route vers le ML Service (port 5000). La réponse JSON contient `prediction`, `probability_pass` et `model_used` — affichés dans le composant Angular dédié au profil étudiant.

Q : Que se passe-t-il au premier démarrage du ML Service ?

R : `ensure_model()` vérifie si `best_classifier.pkl` existe. Si non → il appelle `train_and_save()` qui : charge le CSV, fait le prétraitement, entraîne PCA, K-Means, RF, XGBoost avec CV k=5, compare les deux et sauvegarde tous les `.pkl` dans `/app/models/`. Le service est alors prêt à répondre aux requêtes. Les modèles sont persistés — les redémarrages suivants sont instantanés.

Q : Comment le frontend sait quel modèle est actif ?

R : La réponse de `/predict` inclut le champ `model_used: "xgboost"` ou `"random_forest"`. L'endpoint `GET /model-info` retourne le JSON complet des métriques incluant `best_classifier`, les scores CV de RF et XGBoost, et toutes les métriques d'évaluation. C'est ce que le frontend peut afficher dans un dashboard admin.

10 Questions-Réponses pour la Soutenance

Ces questions sont les plus susceptibles d'être posées. Entraîne-toi à répondre à voix haute — pas à réciter, à expliquer.

Questions sur le Dataset & Prétraitement

Q : Pourquoi ne pas avoir géré le déséquilibre des classes (67% / 33%) ?

R : On a utilisé `stratify=y` dans le train/test split et StratifiedKFold en CV, ce qui garantit la même proportion dans chaque fold. Des techniques comme SMOTE (sur-échantillonnage) ou `class_weight` auraient pu améliorer le recall sur la classe "échec", mais les métriques obtenues ($F1=0.889$, $AUC=0.897$) sont satisfaisantes sans. C'est un choix de simplicité justifié par les résultats.

Q : Quelles features ont le plus d'influence sur la prédiction ?

R : G1 et G2 (notes des trimestres précédents) ont la plus forte corrélation avec G3 (>0.8). `failures` (échecs passés) est fortement corrélée négativement. Dans Random Forest, on peut calculer l'importance des features — G1 et G2 dominant, suivies de failures et absences. XGBoost confirme ce classement. La PCA fusionne ces corrélations dans les premières composantes principales.

Questions sur les Algorithmes

Q : Pourquoi ne pas utiliser SVM ou Logistic Regression ?

R : On a choisi Random Forest et XGBoost car : (1) ils gèrent naturellement les données non-linéaires sans transformation, (2) ils sont robustes aux outliers, (3) ils fournissent des probabilités calibrées via `predict_proba()` nécessaires pour l'API. SVM serait viable mais plus difficile à interpréter et à déployer. La Logistic Regression est trop linéaire pour capturer les interactions complexes entre features socio-démographiques.

Q : Qu'est-ce que le ROC-AUC et comment l'interpréter ?

R : La courbe ROC trace le taux de vrais positifs (TPR) contre le taux de faux positifs (FPR) pour tous les seuils de décision (0 à 1). L'AUC est l'aire sous cette courbe. $AUC=0.5$ = modèle aléatoire. $AUC=1$ = modèle parfait. Notre XGBoost à $AUC \approx 0.897$ signifie : si on prend un étudiant qui va réussir et un qui va échouer au hasard, le modèle attribuera la probabilité de réussite la plus haute au bon étudiant dans 89.7% des cas.

Q : Pourquoi XGBoost est-il "meilleur" que Random Forest sur ce problème ?

R : XGBoost optimise directement la fonction de perte (logloss) par descente de gradient, ce qui lui permet de corriger itérativement ses erreurs. Random Forest fait un vote égal de 100 arbres indépendants — tous les arbres ont le même poids. XGBoost donne plus de poids aux arbres qui corrigent les erreurs difficiles. Sur un dataset à 395 exemples, XGBoost profite mieux de la petite taille en ajustant finement les résidus. Cela se traduit par un AUC CV de 0.897 vs 0.891 pour RF.

Q : Si les scores CV de RF et XGBoost étaient identiques, lequel choisir ?

R : Random Forest, car il est plus simple (moins d'hyperparamètres), plus rapide à entraîner, et plus stable. Le principe de parcimonie (Occam's razor) dit : entre deux modèles de même performance, choisir le plus simple. Notre code gère ce cas : `if xgb_auc_cv >= rf_auc_cv → xgb else rf` — en cas d'égalité parfaite, XGBoost est légèrement favorisé, mais dans la pratique l'un des deux sera toujours strictement meilleur.

Questions sur le Déploiement Web**Q : Comment l'API Gateway sait qu'elle doit router vers le ML Service ?**

R : Dans la configuration de l'API Gateway, on définit une route : les requêtes vers `/api/ml/**` sont redirigées vers `ml-service:5000`. Le ML Service est enregistré sur Eureka sous le nom "ml-service". C'est le même mécanisme que pour les autres services Spring Boot — le ML Service FastAPI s'intègre comme n'importe quel autre microservice.

Q : Le modèle est-il rechargé à chaque requête ?

R : Non. Les modèles sont chargés UNE SEULE FOIS au démarrage de l'application dans le handler `lifespan()` de FastAPI, et stockés en mémoire dans le dictionnaire `_models`. Chaque requête `/predict` accède directement à `_models["best"]` en mémoire — pas de rechargement disque. C'est pour ça que les premières prédictions après démarrage sont instantanées.

Q : Comment assurer la cohérence entre le prétraitement à l'entraînement et à l'inférence ?

R : C'est un problème critique en ML en production. On sauvegarde le MÊME objet scaler (ajusté sur le train set) dans `scaler.pkl` et le MÊME objet pca dans `pca.pkl`. À l'inférence, on appelle `scaler.transform()` (pas `fit_transform()` !) avec le scaler déjà ajusté. Idem pour PCA. Si on refittait sur les données d'inférence, les transformations seraient différentes et les prédictions incohérentes. Les encodeurs LabelEncoder sont aussi sauvegardés dans `encoders.pkl` pour la même raison.

Questions sur la Contribution MLA

Q : Quel est le lien concret entre ce module ML et la plateforme éducative ?

R : La plateforme gère des demandes de rattrapage (claim-service + retake-service). Avant qu'un étudiant soumette une réclamation pour un rattrapage, le système peut déjà prédire sa probabilité de réussite. Si `probability_pass = 0.18`, le système peut alerter l'enseignant proactivement. Le clustering K-Means permet de créer des groupes d'étudiants pour des actions collectives (soutien ciblé au groupe "Average"). L'endpoint `/analyze` retourne à la fois la prédiction et le profil en un seul appel — conçu pour l'UX Angular.

Q : Quelle est la limite principale de ce modèle ?

R : Le dataset a 395 étudiants portugais de 2008 — il n'est pas représentatif de tous les contextes éducatifs. Le modèle pourrait ne pas généraliser à des étudiants tunisiens ou à un autre curriculum. Le Silhouette Score faible (0.0655) montre que les clusters ne sont pas très séparés. En production, il faudrait ré-entraîner sur des données de la plateforme réelle. C'est ce que l'architecture permet : remplacer `student-mat.csv` par les données réelles et relancer l'entraînement.

11 Checklist Finale Avant Validation

À déposer sur Blackboard

- ☐ Notebook `m1a_student_performance.ipynb` avec toutes les cellules exécutées (outputs visibles)
- ☐ Dataset `student-mat.csv`
- ☐ Tableau récapitulatif BO/DSO/Algorithme/Membre/Conclusion (section 1 du notebook)
- ☐ Ce rapport PDF (MLA_Rapport_Student_Performance.pdf)

À vérifier sur ta machine avant la séance

- ☐ L'application web (Angular) démarre sur `localhost:4200`
- ☐ Le ML Service FastAPI répond sur `localhost:5000/health` → `{"status": "ok"}`
- ☐ Le notebook tourne du début à la fin sans erreur (kernel restart + run all)
- ☐ La cellule XGBoost (section 6b) s'exécute — `xgboost` installé dans l'environnement
- ☐ La cellule Cross Validation (section 6c) produit le graphique de comparaison
- ☐ La cellule summary finale (section 7) affiche le tableau avec les vraies métriques calculées
- ☐ `GET /model-info` retourne les métriques avec `best_classifier` renseigné
- ☐ `POST /predict` avec un étudiant test retourne `prediction + model_used`

À savoir expliquer oralement

- ☐ Pourquoi PCA avant K-Means et Random Forest ?
- ☐ Différence RF vs XGBoost (parallèle vs séquentiel)
- ☐ Pourquoi StratifiedKFold et pas KFold simple ?
- ☐ Pourquoi ROC-AUC comme critère de sélection du meilleur modèle ?
- ☐ Comment les encodeurs et scaler sont sauvegardés et réutilisés à l'inférence ?
- ☐ Comment l'API Gateway route vers le ML Service ?

☐ Lien entre ce module ML et les demandes de rattrapage de la plateforme

Le module MLA est complet : PCA + K-Means + Random Forest + XGBoost + Cross Validation déployés sur l'application web. Tu es prêt(e) pour la validation.